

AI FITTRACK

PERSONALIZED FITNESS RECOMMENDATIONS POWERED BY AI

PHASE 5 – PROJECT DEVELOPMENT

CONTENTS

1. DEVELOPMENT OVERVIEW
2. PROJECT FOLDER CREATION
3. NODE.JS PROJECT AND DEPENDENCY SETUP
4. ENVIRONMENT CONFIGURATION
5. SERVER DEVELOPMENT
6. BACKEND STRUCTURE AND MODULE DEVELOPMENT
7. AUTHENTICATION DEVELOPMENT
8. WORKOUT DEVELOPMENT
9. DATABASE DEVELOPMENT
10. AI INTEGRATION DEVELOPMENT
11. MIDDLEWARE AND ERROR HANDLING
12. REST API DEVELOPMENT
13. VERIFIED LOCAL EXECUTION
14. DEVELOPMENT EVIDENCE AND SKILL WALLET ALIGNMENT
15. DEVELOPMENT CONCLUSION

1. DEVELOPMENT OVERVIEW

The Project Development phase documents the actual implementation and configuration of the assigned AI FitTrack backend. The supplied project is a Node.js and Express.js REST API using MongoDB/Mongoose, JWT authentication, bcryptjs password protection, and Google Gemini AI integration.

The implementation is organized into configuration, controllers, middleware, models, routes, services, and application/server files. The development record below follows the supplied project structure rather than introducing a replacement architecture.

2. PROJECT FOLDER CREATION

The supplied project is organized as the AI-FitTrack backend project. The actual verified source uses a src directory for application code.

```
AI-FitTrack/
├── .env
├── .env.example
├── .gitignore
├── FitTrack.postman_collection.json
├── package.json
├── package-lock.json
├── README.md
├── src/
│   ├── app.js
│   ├── server.js
│   ├── config/
│   │   └── db.js
│   ├── controllers/
│   │   ├── aiController.js
│   │   ├── authController.js
│   │   └── workoutController.js
│   ├── middleware/
│   │   ├── auth.js
│   │   └── errorHandler.js
│   ├── models/
│   │   ├── User.js
│   │   └── Workout.js
│   ├── routes/
│   │   ├── aiRoutes.js
│   │   ├── authRoutes.js
│   │   ├── index.js
│   │   └── workoutRoutes.js
│   └── services/
│       └── geminiService.js
```

This structure separates HTTP routing, business processing, authentication, database models, database configuration, and AI service responsibilities.

3. NODE.JS PROJECT AND DEPENDENCY SETUP

The project is a Node.js application managed through npm. The supplied package configuration includes scripts for starting the server directly and running it through Nodemon during development.

Package / Tool	Development Purpose
Node.js	Backend JavaScript runtime.
Express.js	REST server, routing, and middleware.
Mongoose	MongoDB object data modeling and schema validation.
dotenv	Loads environment variables.
bcryptjs	Password hashing and password comparison.
jsonwebtoken	JWT creation and verification.
cors	Cross-origin request support.
morgan	HTTP request logging.
@google/genai	Google Gemini API integration.
Nodemon	Development-time automatic server restart.

The verified local project was installed successfully with npm. The runtime used during execution was Node.js v24.19.0 and npm 11.17.0.

4. ENVIRONMENT CONFIGURATION

Sensitive configuration is supplied through environment variables rather than hard-coded application values. The project uses variables for the server port, MongoDB connection, JWT secret, Gemini API key, and Gemini model configuration.

```
PORT=5000
MONGO_URI=<MongoDB connection string>
JWT_SECRET=<JWT secret>
GEMINI_API_KEY=<Gemini API key>
GEMINI_MODEL=<configured Gemini model>
```

Sensitive configuration is excluded from the repository and project evidence. The .env.example file provides the safe environment-configuration template, while the actual .env file contains private configuration values.

5. SERVER DEVELOPMENT

server.js is the application entry point. It loads environment configuration, connects to MongoDB through the database configuration module, imports the Express application, and starts the HTTP server.

The Express application in app.js configures JSON request parsing, CORS, Morgan logging, the health endpoint, API route mounting, a not-found response, and centralized error handling.

```
const app = express();
app.use(cors());
```

```

app.use(express.json());
app.use(morgan('dev'));

app.get('/', (req, res) => {
  res.json({
    success: true,
    message: 'AI FitTrack API is running successfully',
    version: '1.0.0'
  });
});

```

The API route group is mounted under /api, with separate route modules for authentication, workouts, and AI operations.

6. BACKEND STRUCTURE AND MODULE DEVELOPMENT

Module	Actual Files	Development Responsibility
Application	src/app.js, src/server.js	Express configuration, middleware, health endpoint, route registration, database startup, and server execution.
Configuration	src/config/db.js	MongoDB/Mongoose connection using MONGO_URI.
Authentication	authRoutes.js, authController.js, middleware/auth.js, User.js	Registration, login, profile retrieval, JWT verification, password hashing/comparison.
Workout	workoutRoutes.js, workoutController.js, Workout.js	Workout creation, retrieval, search, update, and deletion.
AI	aiRoutes.js, aiController.js, services/geminiService.js	Workout recommendation and fitness-insight requests through Google Gemini.
Error Handling	middleware/errorHandler.js	Centralized handling of application errors and failed requests.

7. AUTHENTICATION DEVELOPMENT

The authentication module provides registration, login, and profile access. Registration creates a user through the User model, while the model hashes the password before persistence. Login verifies the supplied password and returns a JWT token.

The authentication middleware reads the Bearer token from the Authorization header, verifies the token using the configured JWT secret, loads the associated user without returning the password, and attaches the authenticated user to req.user.

Endpoint	Purpose
POST /api/auth/register	Create a user account.
POST /api/auth/login	Authenticate the user and return a JWT token.
GET /api/auth/profile	Return the authenticated user's profile.

```
const token = jwt.sign(
  { id: user._id },
  process.env.JWT_SECRET,
  { expiresIn: '30d' }
);
```

The supplied implementation uses a 30-day JWT expiration period.

8. WORKOUT DEVELOPMENT

The Workout model represents workout records associated with a User. The implemented fields include workoutName, category, duration, caloriesBurned, workoutDate, and the user reference, together with Mongoose timestamps.

Operation	Endpoint	Implementation
Create	POST /api/workouts	Creates a workout associated with the authenticated user.
Retrieve all	GET /api/workouts	Returns workout records for the authenticated user.
Search	GET /api/workouts/search	Searches by name, category, or date.
Retrieve by ID	GET /api/workouts/:id	Retrieves a specific workout after ownership checking.
Update	PUT /api/workouts/:id	Updates an existing workout.
Delete	DELETE /api/workouts/:id	Deletes an existing workout after ownership checking.

The search route is defined before the /:id route so that the search path is interpreted correctly. Workout retrieval, update, and deletion are user-scoped through the authenticated user context.

9. DATABASE DEVELOPMENT

MongoDB Atlas is used for the verified execution environment. Mongoose provides the User and Workout schemas and handles the application-to-database connection.

```
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI);
    console.log(`MongoDB Connected: ${conn.connection.host}`);
  } catch (error) {
    console.error(`Error connecting to MongoDB: ${error.message}`);
    process.exit(1);
  }
};
```

Model	Important Fields / Rules
User	name, email, password, createdAt, updatedAt; email is unique and lowercase; password is hashed.
Workout	user reference, workoutName, category, duration, caloriesBurned, workoutDate, createdAt, updatedAt; category and numeric fields are validated.

The verified database scope contains two collections/models: Users and Workouts. No additional Progress, Goals, History, Notifications, or other fitness collections are claimed as part of the supplied implementation.

10. AI INTEGRATION DEVELOPMENT

AI functionality is separated into AI routes, an AI controller, and the Gemini service. The controller receives validated fitness inputs and delegates the external model request to `geminiService.js`.

AI Function	Inputs	Purpose
Workout Recommendation	age, fitnessGoal, experience	Generate personalized workout guidance.
Fitness Insights	totalWorkouts, averageDuration, totalCaloriesBurned	Generate performance analysis, improvement suggestions, motivational advice, and progress-oriented insight.

```
const { GoogleGenAI } = require('@google/genai');
const ai = new GoogleGenAI({
  apiKey: process.env.GEMINI_API_KEY
});
```

The Gemini model is configured through the environment rather than being duplicated across service functions. This keeps model configuration centralized and allows the configured model to be changed without exposing credentials in source code.

11. MIDDLEWARE AND ERROR HANDLING

The supplied implementation uses authentication middleware to protect secured routes and centralized error handling for unexpected application errors. Express also provides a not-found path for unmatched routes.

Middleware / Mechanism	Purpose
CORS	Allows configured cross-origin requests.
<code>express.json()</code>	Parses JSON request bodies.
Morgan	Logs HTTP requests during execution.
JWT auth middleware	Verifies protected-route authentication.
Centralized error handler	Provides consistent handling of application errors.
Not-found handling	Returns a response for unsupported API paths.

12. REST API DEVELOPMENT

The backend exposes RESTful operations using GET, POST, PUT, and DELETE methods. Authentication, workout management, search, and AI operations are separated into route modules and mounted under the /api route group.

Module	Method	Endpoint
Authentication	POST	/api/auth/register
Authentication	POST	/api/auth/login
Authentication	GET	/api/auth/profile
Workout	POST	/api/workouts
Workout	GET	/api/workouts
Workout	GET	/api/workouts/search
Workout	GET	/api/workouts/:id
Workout	PUT	/api/workouts/:id
Workout	DELETE	/api/workouts/:id
AI	POST	/api/ai/workout-recommendation
AI	POST	/api/ai/fitness-insights

13. VERIFIED LOCAL EXECUTION

During the project execution, npm installation completed successfully. MongoDB Atlas was used after the local MongoDB connection was unavailable. The application subsequently connected successfully to MongoDB Atlas and started on port 5000.

Server running in development mode on port 5000

MongoDB Connected: <Atlas host>

The root health endpoint was verified at <http://localhost:5000> and returned a successful AI FitTrack API response with version 1.0.0.

```
{
  "success": true,
  "message": "AI FitTrack API is running successfully",
  "version": "1.0.0"
}
```

The execution evidence consists of the server terminal output and successful health response, with environment variables, passwords, API keys, and other credentials excluded.

14. DEVELOPMENT EVIDENCE AND SKILL WALLET ALIGNMENT

Skill Wallet Task	Development Evidence to Retain
7. Creating Project Folder	Project folder and source structure.
8. Server Setup	npm installation, environment setup, server startup, and health response.
9. Backend Structure	VS Code project tree showing src/config, controllers, middleware, models, routes, and services.
10. Development and Explanation	Selected source-code sections and explanations of application, authentication, workout, AI, middleware, and error-handling modules.
11. Configure MongoDB	MongoDB Atlas configuration evidence with credentials redacted.
12. Create Database Connection	db.js implementation and successful MongoDB connection evidence.
13. Create Schema and Models	User.js and Workout.js schema evidence.
14. Steps for Project Execution	Documented installation, configuration, startup, and API testing procedure.

15. DEVELOPMENT CONCLUSION

The Project Development phase records the actual AI FitTrack implementation as a modular Node.js and Express.js backend. The developed system separates authentication, workout management, database access, AI integration, middleware, and error handling into dedicated components.

The implementation baseline is now ready for the Project Testing phase. Testing documentation will focus on the verified server environment, authentication APIs, workout CRUD and search operations, AI endpoints, error responses, regression checks, and final test evidence.